面向 2.5D 系统集成的芯粒布局方法

CNN–Contextual Bandit 算法组合实验设计方案

——基于状态表征、宏动作选择与统一物理反馈的多算法布局优化

本方案的核心不是把 SA、强化学习和解析法串成流水线，而是学习一个**状态条件化的算法组合策略**：

布局状态 $\longrightarrow$ CNN 上下文表征 $\longrightarrow$ 选择算法/配置/预算 $\longrightarrow$ 物理评估反馈。

文档定位：正式实验前的实验设计、接口定义、消融方案和开源实现清单。

当前方法版本：第一版采用 2D CNN + 有限配置库 + Contextual Bandit；第二版扩展到 3D CNN/ConvLSTM。

重要边界：本文档中的性能结论均为待验证的实验假设，不将已有项目的报告数值当作本项目结果。

2026 年 10 月 18 日

目录

1 研究问题与总体判断 **2**
1.1 研究张力 . . . 2
1.2 贡献增量的准确表述 . . . 2

2 总体架构：闭环决策而非流水线拼接 **2**
2.1 系统流程 . . . 2
2.2 统一状态表示 . . . 3

3 分阶段实验设计 **3**
3.1 阶段 0：基准数据、统一格式与物理评估器 . . . 3
3.2 阶段 1：三类算法 Worker 与有限配置库 . . . 4
3.3 阶段 2：算法轨迹、状态转移与训练样本 . . . 5
3.4 阶段 3：2D CNN + Contextual Bandit . . . 5
3.5 阶段 4：3D CNN/ConvLSTM 学习状态流 . . . 6

4 物理评估器设计 **6**
4.1 评估器不是黑盒奖励函数 . . . 6
4.2 三个物理子评估器 . . . 7
4.3 多保真度与公平性 . . . 7

5 对照实验与消融矩阵 **7**
5.1 主实验 . . . 7
5.2 必须控制的变量 . . . 8
5.3 支持或否定核心假设的结果 . . . 8

6 开源实现与复用边界 **8**
6.1 建议的代码目录 . . . 9

7 实施顺序与最小可行版本 **10**

8 预期论文叙事与风险边界 **10**

9 结论 **11**

1 研究问题与总体判断

1.1 研究张力

2.5D 芯粒布局同时受到互连线长、峰值温度、面积利用率、芯粒不重叠和边界合法性等约束。不同算法的优势并不对所有状态保持不变：模拟退火擅长在局部邻域中持续探索，解析法通常具有较强的目标函数驱动和较快的收敛，强化学习能够利用反复交互形成策略，但它们对初始布局、热点分布、连接密度和可行域形状的敏感性不同。

因此，真正需要研究的问题不是“如何让三个算法依次工作”，而是：

> 在给定布局状态和统一物理评估预算下，能否从状态的空间分布与历史变化中预测各个算法配置的短期收益，并在线选择最合适的宏动作？

这里的“agent”应被定义为**算法组合决策器**，而不是自然语言规划器。它不直接替代物理规律，也不直接生成可信的布局坐标；它学习的是“在什么状态下调用哪一种优化器、使用什么配置、运行多大预算”。物理评估器仍然是最终裁决者。

1.2 贡献增量的准确表述

本项目相对于“固定使用一个算法”或“把多个算法首尾连接”的具体增量应写成以下三点。

1. **状态条件化的算法选择**：将当前布局编码为包含线长、热、占用和合法性信息的空间状态张量，Contextual Bandit 根据该状态选择 $(\text{algorithm}, \text{configuration}, \text{budget})$ 宏动作。
2. **统一接口下的可替换布局算子**：SA、解析法和 RL worker 均接收同一种布局状态、输出同一种候选布局，并由同一个 evaluator 返回可比指标；它们是可竞争的动作，不是预先规定的阶段链路。
3. **把“算法偏好”变成可测量的状态转移**：除了记录最终结果，还记录算法作用前后的状态差分、空间影响图和资源成本，从而回答“某算法在何种布局结构上有效、改善了哪个指标、代价是什么”。

论文主线

空间状态表征定义了决策上下文；**Contextual Bandit** 定义了算法选择；**布局 worker** 提供候选动作；**统一物理评估器**提供奖励与约束；**状态流数据**使决策器学习算法的适用条件。整个系统的创新点位于“状态–算法–结果”之间的可学习映射，而不在于简单增加 agent 数量。

2 总体架构：闭环决策而非流水线拼接

2.1 系统流程

每个决策时刻 t 的流程定义为：

$$X_t \xrightarrow{\text{CNN encoder}} z_t \xrightarrow{\text{Contextual Bandit}} a_t = (k_t, c_t, H_t) \xrightarrow{\text{worker } k_t} Y_{t+1} \xrightarrow{\text{Physics Evaluator}} (X_{t+1}, m_{t+1}, r_t), \tag{1}$$

其中 X_t 是当前布局状态，k_t 是算法类别，c_t 是算法配置，H_t 是本次宏动作的预算，Y_{t+1} 是候选布局，m_{t+1} 是合法性和物理指标，r_t 是控制器根据目标函数计算的奖励。只有当候选布局满足硬合法性约束或经过确定性修复后，才允许进入下一次决策。

与原多阶段 Agent 方案的关系

原方案中的 order、thermal、geometry 不再作为三个固定顺序的 agent。它们被改写为 **可学习的诊断通道/动作约束**：order 结构、热热点和几何拥挤分别进入状态张量与 wiremask；最终实际执行的是一个算法 worker。若实验发现某类状态稳定需要先做热修复，再可把“热修复宏动作”加入动作库，但这个顺序应由对照实验支持，而不是在系统结构中预先假定。

2.2 统一状态表示

对于给定的 chiplet 集合和芯片边界，将布局栅格化为 $H \times W$，第一版可取 256×256。输入不是单张图，而是多通道张量：

$$X_t = [M_{\text{occ}}, M_{\text{power}}, M_{\text{conn}}, M_{\text{thermal}}, M_{\text{wire}}, M_{\text{legal}}, M_{\text{order}}, v_t]. \tag{2}$$

- M_{occ}：芯粒占用、边界和重叠信息；
- M_{power}：功耗投影或功耗密度；
- M_{conn}：与网络连接或邻近连接需求相关的空间投影；
- M_{thermal}：当前热图或快速热模型输出；
- M_{wire}：线长敏感区域或连接拥挤度；
- M_{legal}：动作可行域；
- M_{order}：芯粒顺序/优先级投影；
- v_t：全局标量，如总 HPWL、峰值温度、重叠面积、利用率、剩余 evaluator 预算和历史动作。

wiremask 的位置。 本设计中 wiremask 不是额外的“第四个优化阶段”，也不是只在最后筛选区域。它有两个作用：

1. 作为 CNN 的结构化输入，使 Bandit 看到“当前最需要被改变的空间区域”；
2. 作为 worker 的动作约束，将候选移动、局部搜索或 RL 动作限制在允许区域/允许方向内，减少无效探索。

第一版不要求 wiremask 具有端到端可学习的生成器。建议先用确定性规则生成 baseline mask，再把 mask 的阈值、膨胀半径和指标权重加入配置库，作为可控消融因素。

3 分阶段实验设计

3.1 阶段 0：基准数据、统一格式与物理评估器

目标。 在任何 Bandit 训练前，先建立唯一的 canonical layout representation，避免三个算法各自使用不同的坐标、线长定义或热评价器，导致结果不可比。

建议的数据结构如下：

字段	内容
几何	chiplet ID、宽高、方向、坐标、die 边界、禁止区域、间距约束
连接	net 列表、pin offset、net weight、芯粒连接矩阵
热	功耗、材料/层配置、散热边界条件、热模型版本
状态	栅格化通道、全局指标、当前预算、历史动作和随机种子
输出	候选布局、合法性、HPWL、路由线长、峰值/平均温度、运行时间、调用次数

阶段 0 的验收。 对每个基准 case，给定同一个布局文件，三种 worker 的 evaluator 输出应在容差内一致；至少测试边界、重叠、旋转、pin offset、HPWL 和功耗读取。若同一布局在三个 worker 中得到不同 HPWL，不进入 Bandit 训练，先修正 adapter。

3.2 阶段 1：三类算法 Worker 与有限配置库

第一版不要同时进行连续超参数优化。将算法和关键参数离散成有限配置库，使 Bandit 先解决一个定义清晰的有限动作选择问题：

$$\mathcal{A} = \{(k, c) : k \in \{\mathrm{SA}, \mathrm{AN}, \mathrm{RL}\}, c \in \mathcal{C}_k\}. \tag{3}$$

其中 AN 表示 analytical placement。每个 arm 在固定的 evaluator 预算 H 下运行一次，返回宏观结果，不把内部每一个随机移动都交给 Bandit。

Worker	第一版配置字段	推荐的第一批离散配置
SA	初温、降温率、每温度迭代数、移动半径、wire/thermal 权重、局部/全局搜索比例	低/中/高三档温度；快/慢两档降温；小/大移动半径；线长主导/热主导/均衡三种权重
AN	wire 权重、thermal 权重、密度权重、学习率、温度阈值、热感知开关	WL-driven、Thermal-aware、两种热权重、两种密度权重，组成不超过 8–12 个配置
RL	checkpoint、动作步长、熵系数、折扣因子、候选动作半径、热模型版本	预训练 policy 的小/中/大动作半径；低/中探索；固定 policy 与短 horizon fine-tune 两档

为什么先用有限配置库。 当前研究问题是“状态是否能指导算法选择”，不是“连续超参数优化是否有效”。有限 arm 可以计算选择 regret、top-k 命中率和每个状态簇的算法偏好。如果一开始同时做算法选择和连续参数优化，性能提升无法归因，且会把 Bandit、Bayesian optimization 和 worker 内部训练混在一起。

连续参数的后续版本。 第二阶段可采用分层动作：第一层选择算法 k，第二层在该算法的可信参数范围内用离散化、Gaussian Process Bandit 或 Thompson Sampling 优化连续参数 θ。此时必须将

参数搜索成本纳入 reward，且保留有限库版本作为基线。

3.3 阶段 2：算法轨迹、状态转移与训练样本

轨迹与状态流的区别。 三种数据对象必须分开命名：

对象	定义与用途
算法轨迹	worker 内部的完整事件序列，如 SA 的每次移动/接受概率，RL 的每步 observation/action，解析法的每次迭代。它保留细节，适合调试和重放。
宏观状态转移样本	一次宏动作前后记录 $(X_t, a_t, X_{t+1}, r_t, c_t)$，其中 a_t 是算法、配置和预算，c_t 是时间/evaluator 调用等成本。它是 Bandit 的基本训练单元。
状态流	将连续快照抽象为序列 $X_t \to X_{t+1} \to \cdots$，重点学习某种算法改变空间分布和指标的规律，而不关心每一次内部事件。它是 3D CNN/ConvLSTM 的输入。

也就是说，轨迹是“发生过哪些内部事件”，状态流是“状态如何变化”。第一版只需保存宏观状态转移样本；第二版再使用固定时间间隔的连续状态快照构造状态流。

数据采集协议。

1. 按 benchmark instance 划分训练、验证和测试，不能将同一个 case 的不同中间状态随机分到不同集合，以免泄露布局结构。
2. 对每个初始布局，让所有 arm 在同一 evaluator 预算、相同随机种子集合和相同输入状态下独立运行。
3. 每个宏动作保存起点快照、终点快照、增量 mask、指标变化、运行时间、evaluator 调用次数和 worker 配置。
4. 早期必须执行一轮全臂采样，避免 Bandit 只看到先验偏好的算法；后续再使用 epsilon-greedy 或 Thompson Sampling 探索。

3.4 阶段 3：2D CNN + Contextual Bandit

模型。 第一版使用共享 CNN encoder：

$$z_t = f_\phi(X_t), \qquad \hat{q}_t(a) = g_\psi([z_t, e_a]), \tag{4}$$

其中 e_a 是算法/配置的 embedding，$\hat{q}_t(a)$ 预测执行 arm a 后的短期收益。共享 encoder 的意义是让不同算法基于同一个物理状态空间竞争，避免训练三个互不对齐的 CNN。

推荐顺序：

1. 先离线训练 CNN 回归器，预测固定预算 H 下的 ΔHPWL、ΔTpeak、合法性和运行成本；
2. 再将 CNN 表征接入 LinUCB 或 linear Thompson Sampling；
3. 最后做 online update，使用真实 evaluator 返回的 reward 修正预测。

奖励。 不直接把单一的“总分”当作唯一事实，而是记录原始指标并由实验配置产生目标化奖励：

$$r_t = -\lambda_w \Delta\widetilde{WL}_t - \lambda_T \Delta\widetilde{T}_t - \lambda_A \Delta\widetilde{A}_t - \lambda_L \mathbb{1}[\text{illegal}] - \lambda_C \widetilde{C}_t. \tag{5}$$

其中 Δ 表示相对当前状态的变化，$\widetilde{C}_t$ 是归一化的运行时间或 evaluator 调用成本。所有 arm 使用相同的成本口径；否则 Bandit 可能只学会选择快而不一定有效的 worker。

Bandit 的评价。 第一版不只报告最终布局分数，还报告：

- arm 选择 regret：与事后知道最优 arm 的 oracle 比较；
- top-1/top-3 选择准确率：预测收益最高的 arm 是否真正有效；
- reward 预测误差和校准误差；
- 在相同 evaluator budget 下的最终 HPWL、峰值温度、合法率和 wall-clock time。

3.5 阶段 4：3D CNN/ConvLSTM 学习状态流

第二版不是“把 2D CNN 换成更大的网络”，而是改变学习目标：预测某个算法宏动作造成的状态变化。

给定长度为 L 的快照序列：

$$\mathcal{S}_{t-L+1:t} = [X_{t-L+1}, \ldots, X_t], \tag{6}$$

3D CNN 或 ConvLSTM 估计：

$$\widehat{F}_{k,c}(\mathcal{S}_{t-L+1:t}) \to (\widehat{X}_{t+1}, \widehat{\Delta m}_{t+1}, \widehat{\Delta C}_{t+1}). \tag{7}$$

输入的时间维度刻画“最近几步变化方向”，空间维度刻画热点/拥挤区域。Bandit 的上下文变为 $\widehat{F}_{k,c}$ 对不同候选动作的预测，而不再只依赖当前静态状态。

第二版的必要约束。 只有当第一版证明静态状态不能区分不同算法的适用条件时，才进入 3D/ConvLSTM。否则 3D 模型增加数据量和训练复杂度，却无法证明状态流带来了决策收益。建议使用以下消融：

1. 2D CNN 静态上下文；
2. 2D CNN + 全局历史标量；
3. 3D CNN/ConvLSTM + 状态差分通道；
4. 3D 模型去掉 wiremask 或去掉 thermal 通道。

4 物理评估器设计

4.1 评估器不是黑盒奖励函数

Physics Evaluator 应返回原始物理量、合法性、计算成本和模型版本；奖励由 controller 计算。推荐接口：

```
evaluate(layout, fidelity, case_config) -> {
    legal, overlap, boundary_violation,
    hpwl, routed_wirelength,
    peak_temperature, mean_temperature,
    area_utilization, runtime, evaluator_calls,
    fidelity, evaluator_version
}
```

不能只返回一个 reward，否则无法判断是线长改善、温度恶化还是成本增加导致的结果。

4.2 三个物理子评估器

1. **GeometryEvaluator**：检查边界、重叠、旋转、间距、禁止区域和栅格占用。它必须确定性执行，并在每次宏动作后调用。
2. **WirelengthEvaluator**：内环使用带 pin offset 和 net weight 的 HPWL 或快速中心距离近似；高保真阶段可调用 TAP-2.5D 的路由/MILP 接口。报告 HPWL 与 routed wirelength 两个不同字段。
3. **ThermalEvaluator**：内环使用紧凑解析热模型或快速热模型；定期抽样和最终验证使用 HotSpot wrapper。快速模型只能提供低成本反馈，不能被表述为最终热真实性本身。

4.3 多保真度与公平性

建议定义三种 fidelity：

等级	组成	使用阶段
Fast	几何合法性 + 增量 HPWL + 紧凑热模型	Bandit 训练内环、worker 的大多数中间步
Medium	几何合法性 + HPWL + 更精细热模型	数据采样和候选动作比较
Exact	几何合法性 + 路由线长 + HotSpot 热评价	验证集、最终布局、跨模型校准

公平性要求：

- 同一次比较中，所有算法使用相同 fidelity、最大 wall-clock 或 evaluator call budget；
- 训练 RL policy 的成本和部署时 inference 成本分开记账；
- 快速热模型和 HotSpot 的偏差用独立校准集报告，不在测试集上反复调参；
- 所有结果记录 evaluator 版本、输入单位、随机种子和源码 commit hash。

5 对照实验与消融矩阵

5.1 主实验

实验组	研究问题
固定 SA	单一经典搜索器的性能上限与成本
固定 analytical	目标函数驱动方法的性能与收敛特征
固定 RL	学习型策略单独使用的效果与泛化
随机选择 arm	多算法组合但无状态决策是否有效
全局指标 Bandit	只有标量上下文时，空间状态是否必要
2D CNN + Bandit	主方法：静态空间状态能否改善算法选择
2D CNN + wiremask	mask 作为输入和动作约束是否进一步改善选择与局部修复
3D/ConvLSTM + Bandit	状态流是否带来超过静态状态的增益

5.2 必须控制的变量

比较时固定：benchmark split、初始布局、芯粒尺寸和功耗、目标权重、evaluator fidelity、总预算、随机种子数和最终验证流程。若某算法需要预训练，应分别报告“包含训练成本的端到端结果”和“已训练 worker 的部署结果”，避免将训练资源隐藏在基线之外。

5.3 支持或否定核心假设的结果

核心假设是“布局状态能够预测不同算法的相对短期收益”。

- 若 2D CNN 相比全局指标 Bandit 在相同预算下提高 reward、降低 regret，支持空间上下文有用；
- 若加入 wiremask 后选择准确率与最终指标均改善，支持 mask 作为结构化输入/约束，而不是装饰性模块；
- 若 3D/ConvLSTM 只提高预测精度，却不提高在线决策的最终结果或成本效率，则不能声称状态流学习带来系统收益；
- 若 Bandit 与最强固定算法无显著差异，应将结论限定为“能识别某些状态下的算法偏好”，而不是宣称普遍优于所有单一算法。

6 开源实现与复用边界

下表中的地址来自项目本地归档的 README 与开源代码清单；正式实验必须在运行前记录仓库 commit/hash、许可证和依赖版本。下列“可复用”只表示适合作为实现参考或 baseline，不表示所有代码都可直接作为透明 worker。

表 1: 三类算法、物理评估器及相关代码地址

类别	项目/地址	在本方案中的用途	复用边界
模拟退火	https://github.com/bu-icsg/TAP-2.5D	SA worker、给定布局评价、快速线长与热/路由调用参考	需要处理 HotSpot、CPLEX 和原始输入格式；路由线长和 HPWL 不混用
解析法	https://github.com/Brilight/ATPlace_pub	analytical baseline、Bookshelf/JSON wrapper、HPWL 与热感知参数参考	PlaceFlow 核心/部分评估代码加密；透明实验应保留公开 wrapper，并单独实现可审计的 analytical worker
强化学习	https://github.com/weiweihook/RLPlanner	RL worker、快速热模型接口和 PPO 配置参考	输入格式与 Bookshelf 不同，需要 canonical adapter；README 中的加速/误差为作者报告，需在本项目上重新校准
热模型/布局掩码参考	https://github.com/xunk33333/MaskPlace2.5D	紧凑热模型、thermal data processing、mask 概念参考	核心 placement code 未公开，不能当作完整透明 placement worker
综合布局参考	https://github.com/MicroSystemsDesignLab/STAMP2.5D	2.5D 布局/热约束相关实现和对照参考	先核对与本项目 benchmark、许可证和依赖的兼容性
热评估器	TAP-2.5D 与 ATPlace_pub 中的 HotSpot wrapper；HotSpot 官方源码需按所采用版本固定	Fast/Medium/Exact thermal evaluator 的接口参考与最终验证	快速模型不能替代最终 HotSpot；记录版本、网格、边界条件和材料参数

6.1 建议的代码目录

为避免把第三方代码和新方法混成不可解释的整体，建议实验仓库采用以下分层：

```
experiment/
  canonical/          # Bookshelf/TAP/RLPlanner -> common IR
  evaluator/
    geometry.py       # deterministic legality
```

```
    wirelength.py      # HPWL / routed-WL adapter
    thermal.py         # compact / HotSpot / calibration
  workers/
    sa_tap.py
    analytical_atplace.py
    rl_rlplanner.py
  context/
    rasterizer.py      # X_t and wiremask
    cnn2d.py
    convlstm3d.py
  bandit/
    finite_arms.py
    linucb.py
    thompson.py
  logging/
    trajectory.py
    state_flow.py
    provenance.py
```

第三方 worker 只负责产生候选布局；它们不允许自带另一套最终 reward。所有指标都经过 ‘evaluator/‘，所有选择都经过 ‘bandit/‘，这样才能证明提升来自状态条件化的算法组合，而不是来自某个基线内部的隐含规则。

7 实施顺序与最小可行版本

1. **MVP-0**：只实现 canonical IR、GeometryEvaluator、HPWL、固定热模型和三种 worker 的离线运行接口。
2. **MVP-1**：建立有限 arm 配置库，跑全臂采样，得到每个 case 的宏观状态转移数据。
3. **MVP-2**：训练 2D CNN，先做收益预测，再接 LinUCB/Thompson Sampling；与固定算法、随机选择和全局标量 Bandit 对比。
4. **MVP-3**：加入 wiremask 的输入与动作约束，验证它是否真正改变选择质量和局部修改效果。
5. **MVP-4**：只有当静态上下文存在明确不足时，采集连续状态快照，训练 3D CNN/ConvLSTM，并报告状态流带来的增益。
6. **MVP-5**：最后加入连续超参数优化或分层 Bandit；不得用它替代第一版有限配置库的归因实验。

8 预期论文叙事与风险边界

论文应围绕一个可证伪的主张展开：

> 在统一的 2.5D 芯粒布局表示和物理评估预算下，空间化的布局状态与 wiremask 能够帮助 Contextual Bandit 选择更适合当前状态的布局优化器，从而在质量–成本权衡上优于

固定算法和无状态组合策略。

该主张对应四类证据：空间状态是否提高收益预测、Bandit 是否降低选择 regret、算法选择是否改善最终布局、改进是否在相同 evaluator 资源下成立。不要把“能够调用三个开源算法”写成核心创新，也不要把“3D CNN 能预测状态变化”直接写成“理解了物理规律”。

主要风险及处理方式如下：

- **三种 worker 输入不一致**：先做 canonical adapter，统一坐标、单位和 pin offset。
- **快速热模型偏差**：用 HotSpot 做独立校准和最终验证，不让近似模型单独支撑最终结论。
- **解析法核心不透明**：将 ATPlace_pub 作为外部 baseline，同时实现一个透明的 analytical worker 用于机制消融。
- **Bandit 学到 case ID 而非布局规律**：按实例划分数据，加入跨规模测试和 held-out topology。
- **3D 模型数据量不足**：先完成 2D 版本；3D 版本只使用固定长度的状态窗口，并与 2D 版本共享数据划分。
- **所谓多 agent 只是包装**：最终系统图必须标出唯一的决策变量 a_t、统一 evaluator、状态反馈和对照组，而不是只画多个模块名称。

9 结论

本方案建议将项目的核心从“多 agent 协同布局”收敛为“面向 2.5D 布局状态的算法组合学习”。第一版先把算法选择和连续参数优化拆开，用有限配置库验证 CNN 上下文是否有用；第二版再用 3D CNN/ConvLSTM 验证状态流是否能带来额外决策收益。这样既响应导师提出的“agent 自主选择算法和参数”，又保留你原方案中 wiremask 对布局区域和动作约束的作用，并且每一项创新都能被明确的消融实验检验。